Atividade Prática 1 – Agentes Inteligentes e seu Ambiente

Gustavo Boeck da Silva
Matheus da Silva Dias
CSCDIA00017 – Prof. Guilherme Schardong – UFSM Campus Cachoeira do Sul

07/10/2026

1 Arquitetura

O programa foi feito em Python e resolve as quatro etapas. A etapa é escolhida automaticamente pelos parâmetros do arquivo de entrada. O código tem quatro arquivos:

- `parser.py`: lê o arquivo de configuração (fornecido pelo professor).
- `ambiente.py`: guarda o mapa real. Verifica paredes e limites, aplica a aspiração, conta as células sujas e sorteia sujeira nova. Também funciona como sensor: devolve as células que estão no campo de visão do agente.
- `agente.py`: decide para onde ir. Guarda o que já viu (células observadas, células visitadas e sujeiras encontradas) e calcula caminhos com busca em largura (BFS), que acha o menor caminho desviando das paredes. O agente não lê o mapa direto: só conhece o que os sensores mostraram.
- `main.py`: roda a simulação, controla a bateria e imprime as métricas.

Movimento, aspiração e recarga contam como uma ação cada.

2 Estratégias

Etapa 1. O agente anda em zigue-zague: linhas pares da esquerda para a direita, ímpares da direita para a esquerda. Se a célula está suja, aspira e segue.

Etapa 2. Tem duas fases.

- *Exploração:* o agente segue o zigue-zague até ter visto todas as células do mapa. As sujeiras que aparecem no campo de visão são guardadas numa lista.
- *Limpeza:* implementamos duas estratégias. `ordem_descoberta` limpa na ordem em que as sujeiras foram vistas. `mais_proxima` vai sempre para a sujeira mais perto.

Etapa 3. Antes de cada deslocamento, o agente calcula se tem bateria para ir até o destino, fazer a ação e voltar à estação:

$$\text{energia} = (\text{passos até o destino} + \text{passos do destino até a estação}) \times c_m + c_v$$

O custo c_v só entra se ele for aspirar. Se a bateria não der, ele volta, recarrega e depois segue.

Etapa 4. As paredes (9) são contornadas com BFS. Se o próximo passo do zigue-zague está bloqueado, o agente vai para a próxima célula livre ainda não visitada. Como pode aparecer sujeira nova, ele atualiza a lista a cada passo. Se não conhece mais nenhuma sujeira, volta a percorrer o mapa procurando. Para quando o mapa fica limpo ou ao atingir `MAX-STEPS`.

3 Resultados

Etapa 1. O resultado bate com o esperado: 63 movimentos, 11 aspirações, 74 ações, 11 células limpas e 0 restantes.

Etapa 2.

Tabela 1: Etapa 2 com diferentes raios de visão. Todas as execuções limparam as 11 sujeiras.

Raio	Visitadas	Só observadas	Mov. exploração	Ações `mais_prox.`	Ações `ordem_desc.`
0	64	0	63	74	74
1	55	9	54	86	100
2	46	18	45	78	92
3	37	27	36	68	82

Pergunta 1 Com raio 1, o agente visitou 55 células. Com raio 3, visitou 37. Quanto maior o raio, menos ele precisa andar, porque enxerga as últimas linhas sem passar por elas.

Pergunta 2 Com raio 1, foram 9. Com raio 3, foram 27. Visitadas + só observadas sempre dá 64, o total do mapa.

Comparação. `mais_proxima` usou 14 ações a menos nos raios 1, 2 e 3 (com raio 0 o programa roda como etapa 1). `ordem_descoberta` segue a ordem do zigue-zague, então fica indo e voltando de um lado para o outro do mapa.

Etapa 3.

Tabela 2: Etapa 3: movimentos / recargas / bateria no final.

Custos	Raio	`mais_proxima`	`ordem_descoberta`
$c_m = 0{,}2$, $c_v = 2{,}0$	1	75 / 0 / 63,0%	89 / 0 / 60,2%
$c_m = 1{,}5$, $c_v = 4{,}0$	1	89 / 1 / 21,0%	111 / 2 / 74,0%
$c_m = 1{,}5$, $c_v = 4{,}0$	2	73 / 1 / 42,0%	101 / 2 / 84,0%

Pergunta 1 O agente só sai para um destino se tiver carga para chegar, agir e voltar à estação. Se não tiver, volta antes. Não é a política ótima, mas ele nunca fica sem bateria.

Pergunta 2 Com os custos originais, o agente gasta só 37% da bateria e não recarrega. Com 1,5 por movimento, uma carga dá para uns 66 passos. Aí ele precisa voltar: 1 recarga com `mais_proxima` e 2 com `ordem_descoberta`. Com raio 2, a exploração fica 9 passos mais curta e sobra mais bateria, mas o número de recargas é o mesmo.

Pergunta 3 Calcula o menor caminho até a estação com BFS, multiplica pelo custo do movimento e soma o custo da aspiração, se for aspirar.

Etapa 4. No exemplo (semente 42), o agente viu as 64 células visitando só 49. Apareceu 1 sujeira nova e o mapa terminou limpo. `mais_proxima` usou 105 ações e `ordem_descoberta`, 109. Rodando com as sementes de 1 a 30, a média foi 127 ações com `mais_proxima` e 133 com `ordem_descoberta`. Com `mais_proxima`, o mínimo foi 102 e o máximo, 258.

Pergunta 1 O zigue-zague sozinho não funciona mais. O agente precisa desviar das paredes, e a distância passa a ser o caminho real, não a linha reta. Isso muda a escolha da sujeira mais próxima e o cálculo da bateria.

Pergunta 2 O que o agente sabe fica desatualizado. Ele precisa continuar olhando enquanto anda e, quando acaba a lista, procurar de novo. A tarefa pode não terminar, por isso existe o `MAX-STEPS`.

Pergunta 3 Não. O mapa muda sem que o agente escolha isso.

Pergunta 4 Não. A mesma ação pode dar resultados diferentes, porque a sujeira aparece ao acaso. Com a mesma semente, porém, o resultado se repete: rodamos duas vezes com a semente 42 e deu 105 ações nas duas.

Pergunta 5 O desempenho muda bastante, de 102 a 258 ações, conforme quantas sujeiras aparecem e onde. Na semente 1, `ordem_descoberta` (164 ações) foi melhor que `mais_proxima` (178). Ou

seja, ir sempre para a mais próxima é melhor na média, mas não em todos os casos.

4 Limitações

- A exploração passa por todas as linhas, mesmo com raio grande. Daria para pular linhas e andar menos.
- Quem decide que o mapa ficou limpo é o simulador, que conhece o mapa real. As decisões do agente usam só o que ele viu.
- A regra da bateria é segura, mas não é a mais econômica.

Referências

[1] RUSSELL, S.; NORVIG, P. *Inteligência Artificial*. 3. ed. Rio de Janeiro: Elsevier, 2013.